LESSON 15 · CHEAT SHEET

Functional Programming Principles

Java 8 → Java 21 · results use the shared course dataset
course "today" = 2026-11-07

Key rules

1. **Pure core, I/O at the edges.** Pass dates/Clock in; never `now()` in logic.
2. Records are **shallow**: `List.copyOf` in the compact constructor. 16
3. `unmodifiableList` is a **view**; `List.copyOf` is a copy. 10
4. Withers return new values; no setters.
5. Factories, currying and partial application replace flags and if/else.
6. Compose: `reduce(identity(), Function::andThen)`.
7. Supplier = lazy. Memoize **pure** functions only; never recursive `computeIfAbsent`.

Canonical snippets

1 · Safe record

```
record Statement(String account, List<String> ids) {
  Statement { ids = List.copyOf(ids); }
}
```

2 · Curry and partially apply

```
Function<BigDecimal,
         Function<BigDecimal, BigDecimal>> fee =
    rate -> amount -> money(amount.multiply(rate));
var standard = fee.apply(new BigDecimal("0.0025"));
```

3 · A pipeline from a list of steps

```
steps.stream().reduce(Function.identity(),
                      Function::andThen);
```

API map

API	Use	Since
`Function.andThen / compose`	g(f(x)) / f(g(x))	8
`Function.identity()`	neutral step	8
`UnaryOperator / BiFunction`	steps, 2-arg rules	8
`Predicate.and/or/negate`	combine rules	8
`Predicate.not`	negate a reference	11
`Supplier, orElseGet`	laziness	8
`Clock.fixed / LocalDate.now(clock)`	testable time	8
`List.of / List.copyOf`	immutable lists	9/10
`record` + compact constructor	validated values	16
`Map.computeIfAbsent`	memoize (non-recursive)	8

andThen vs compose (real output)

```
plus2.andThen(times10).apply(1)   30
plus2.compose(times10).apply(1)   12
pipeline(List.of()).apply(1240.50) 1240.50
```

Mini diagram: functional core



Real results from the course code

Call	Result
expiring in 30 days of 2026-11-07	POL-509 (5), POL-505 (13), POL-502 (23), POL-508 (28)
unmodifiable view vs copy	view=[A, B] copy=[A]
orElse vs orElseGet	evaluated 1 vs 0
memoized FX rate	12 calls, 2 real lookups

Worked example / exercise	Result
T023 audit trail	base=2.29, min=2.29, fx=11.43, cap=10.00
fee totals	{SGD=42.95, USD=16.66} on 45 debits
renewal quote POL-502	1240.50 → GOLD 5% → −25 → +GST = 1257.29

Anti-patterns → fixes

Don't	Do
`LocalDate.now()` in logic	date / Clock parameter
record keeps caller's List	`List.copyOf` in constructor
one method, 4 fused rules	small named functions, composed

Don't	Do
`orElse(expensive())`	`orElseGet(this::expensive)`
memoize an impure function	memoize pure functions only
recursive `computeIfAbsent`	get, compute, then put

Lesson 15 · sticky notes

Cut out, stick on your wall, and read one every morning.

Pure core

clock, DB, console
live at the edges

Records are shallow

List.copyOf in the
compact constructor

View ≠ copy

unmodifiableList: view
List.copyOf: copy

Currying

rate -> amount -> fee
fix the rate, reuse

andThen vs compose

andThen: left → right
compose: right → left

identity()

the 0 of composition:
empty pipeline = input

Lazy

orElseGet, not orElse
Supplier for messages

Memoize safely

pure · bounded
no recursive computeIfAbsent